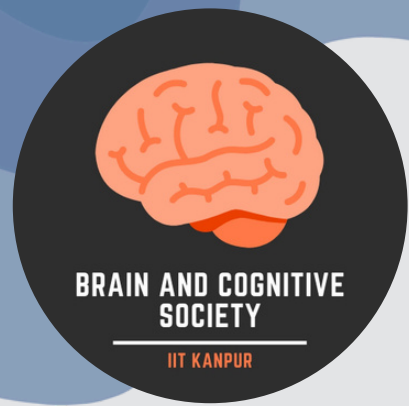
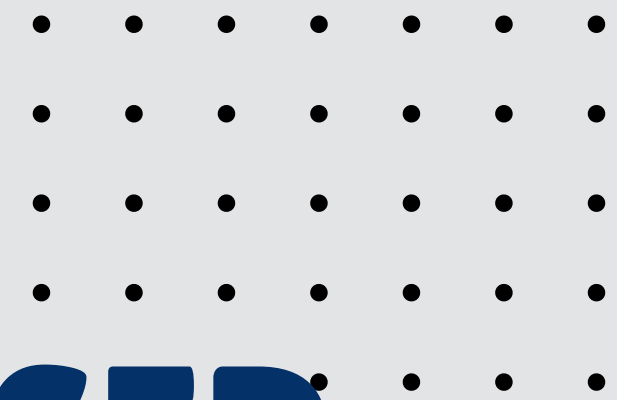


RECALL-E: second half

Introduction to model-based RL approaches.





MODEL-FREE VS. MODEL-BASED

Model-free

Learns *what is good*

$$V(s), Q(s,a), \pi(a|s)$$

Value functions and policies: they tell the agent which actions tend to pay off, but only under the reward that trained them.

TIED TO THE TASK

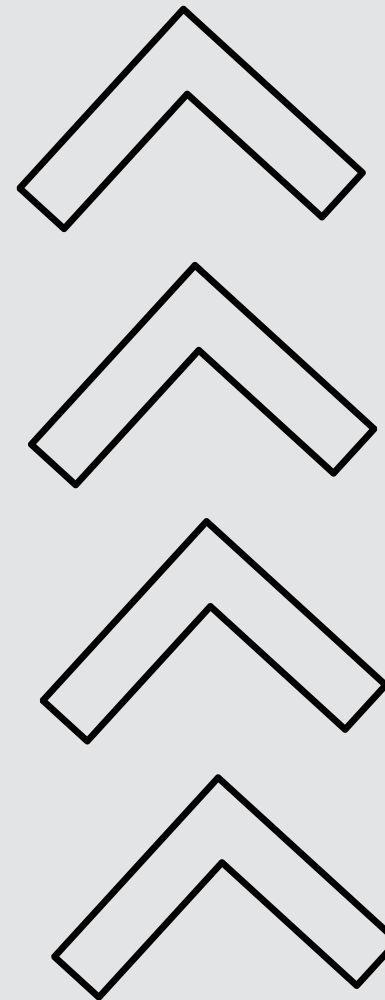
Model-based

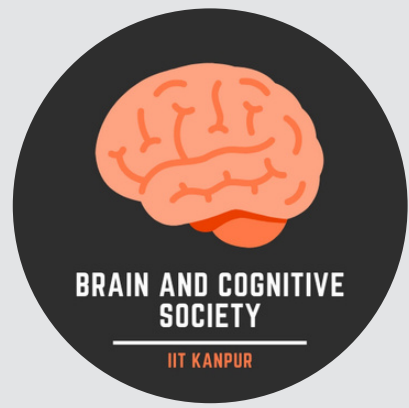
Learns *the environment dynamics*

$$P(s'|s, a), R(s, a, s')$$

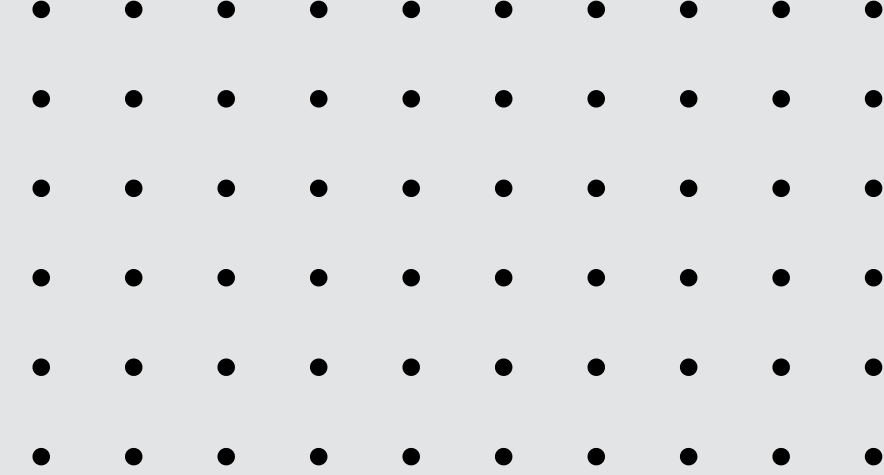
Transition dynamics. The model doesn't know what the task is; it only knows what happens when you take action a when in a given state s

REUSABLE (TASK AGNOSTIC)





THREE SUPERPOWERS(?)



1

Sample efficiency:

Planning inside the model is cheap, parallel, safe. The real environment is the expensive resource while a model can enable us to do thousands of rollouts in a controlled setting.

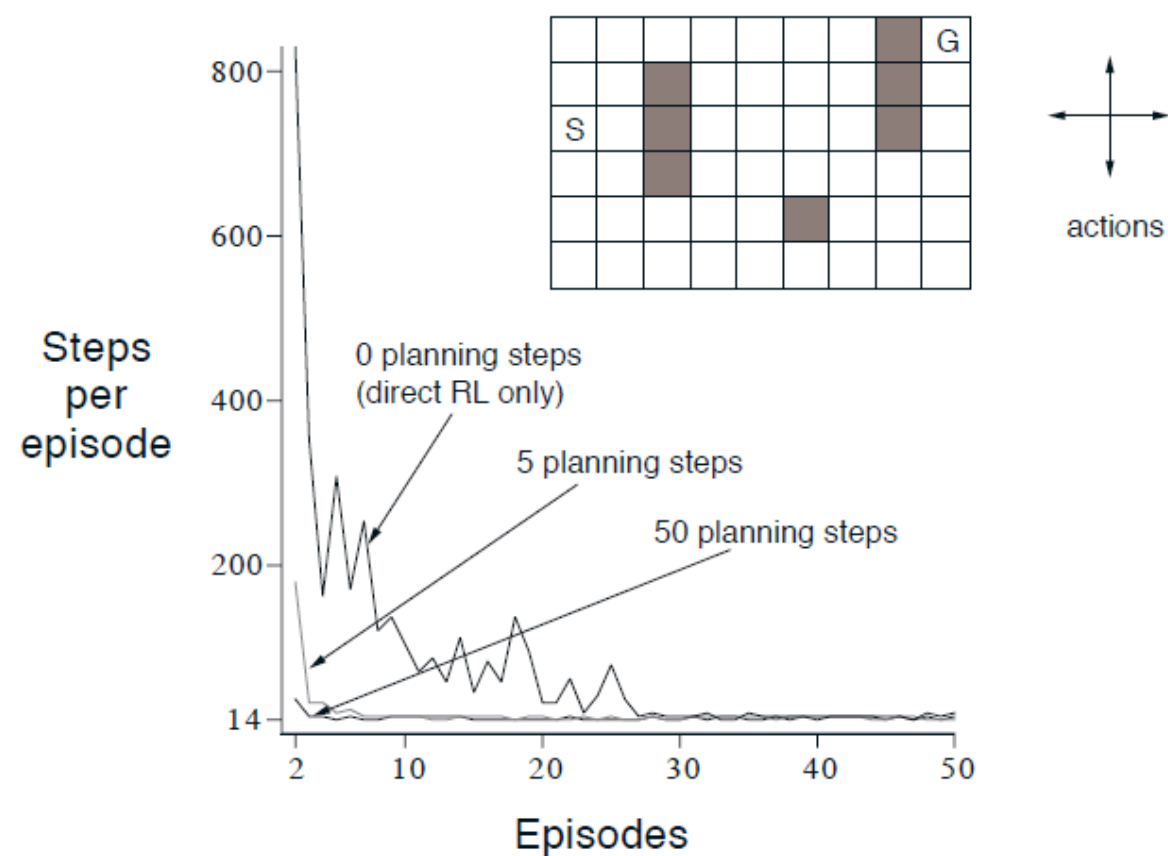
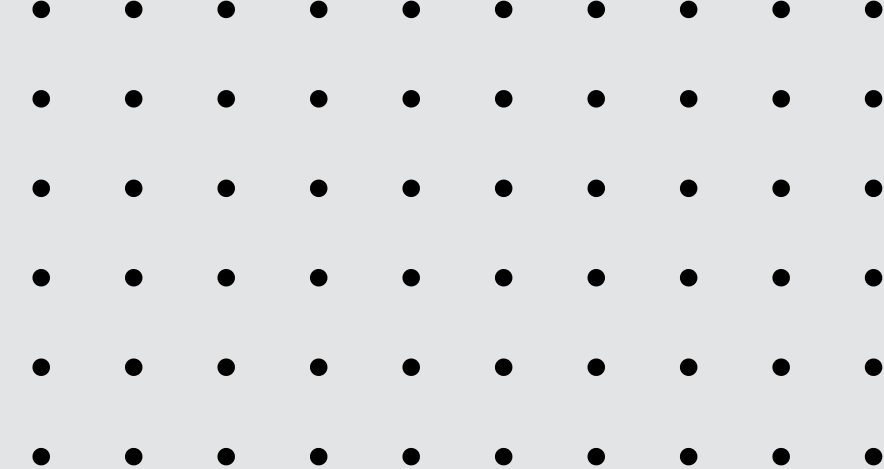
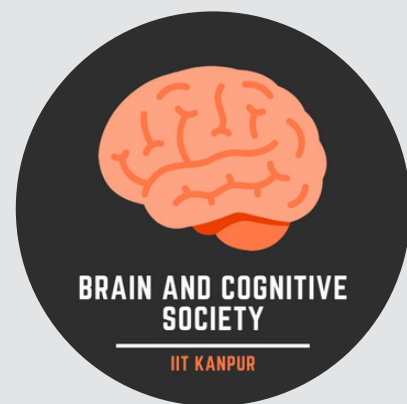


Figure 8.5: A simple maze (inset) and the average learning curves for Dyna-Q agents varying in their number of planning steps (n) per real step. The task is to travel from S to G as quickly as possible.

Dyna-Q at $n=50$: around 3 episodes to solve.
 $n=0$ (no model): around 25 episodes. (S&B Fig 8.5)



Zero-shot transfer:

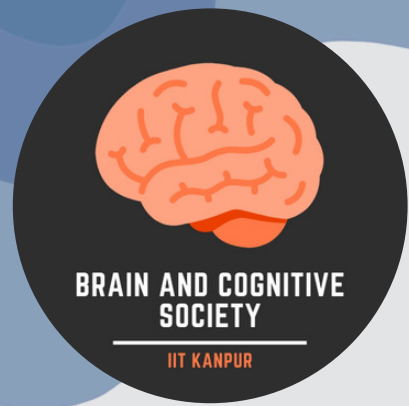
2

In a changing world, an agent's objectives will frequently shift. Under a classic model free paradigm, changing the task goal usually requires substantial retraining. A model-based agent learns how the world works independently of the task ($P(s' | s, a)$). Because the underlying "physics" of the environment (like layout, gravity, or mechanics) remain constant regardless of your current goal, this learned world model is a permanently reusable, task-agnostic asset.

Interpretability:

3

Because the model predicts future states, we can ask "What happens if I do X?" by simulating trajectories before acting. This is in contrast to model free RL which is kinda a "black box" of unknown abstract weights which makes it very difficult to meaningfully debug and pinpoint why a specific action was chosen.

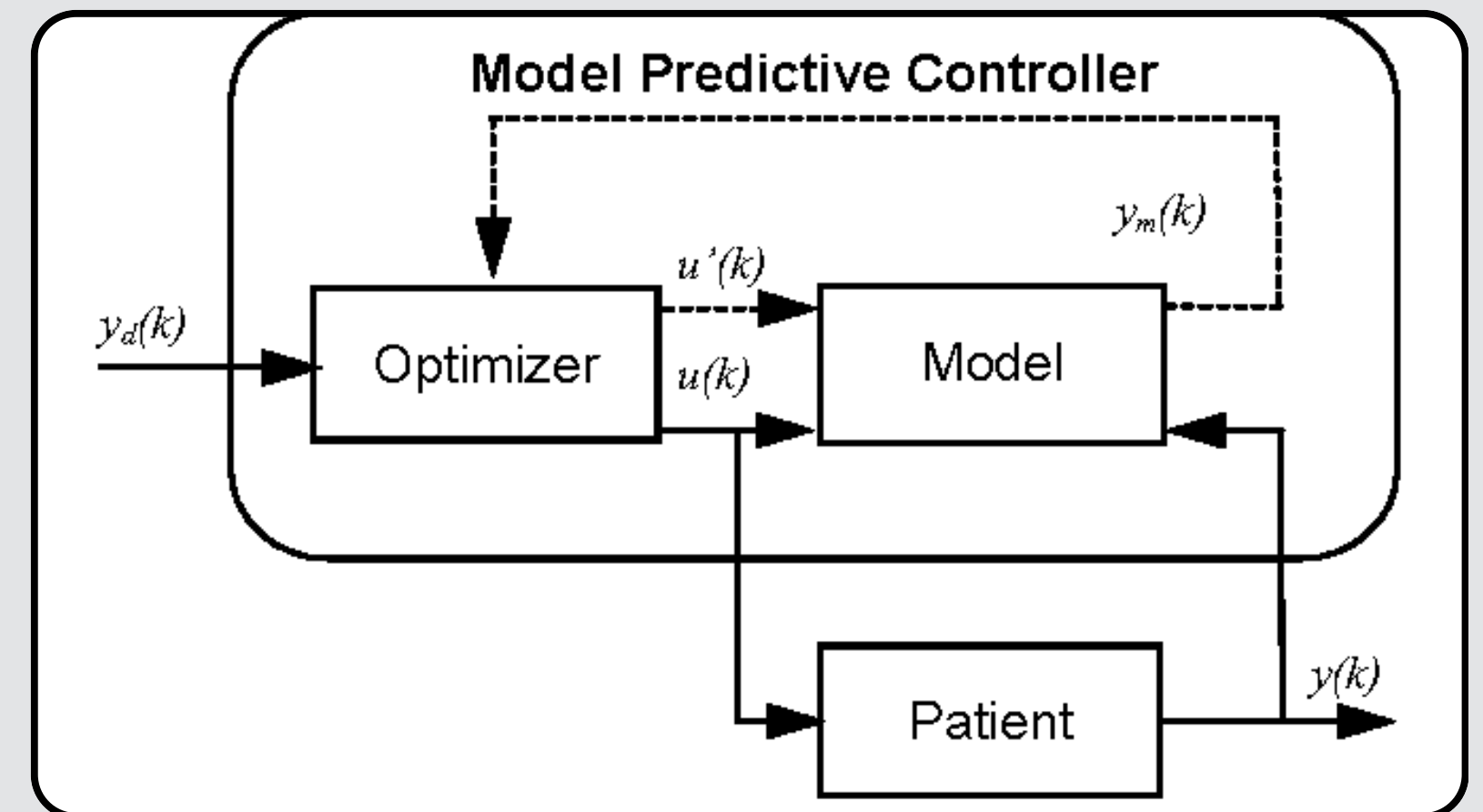


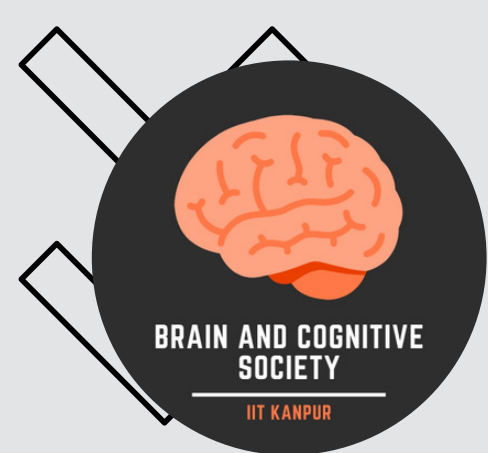
SHIFT TO ONLINE PLANNING

We will replace the value function with a short-horizon optimizer that re-plans every step.

THE MPC LOOP

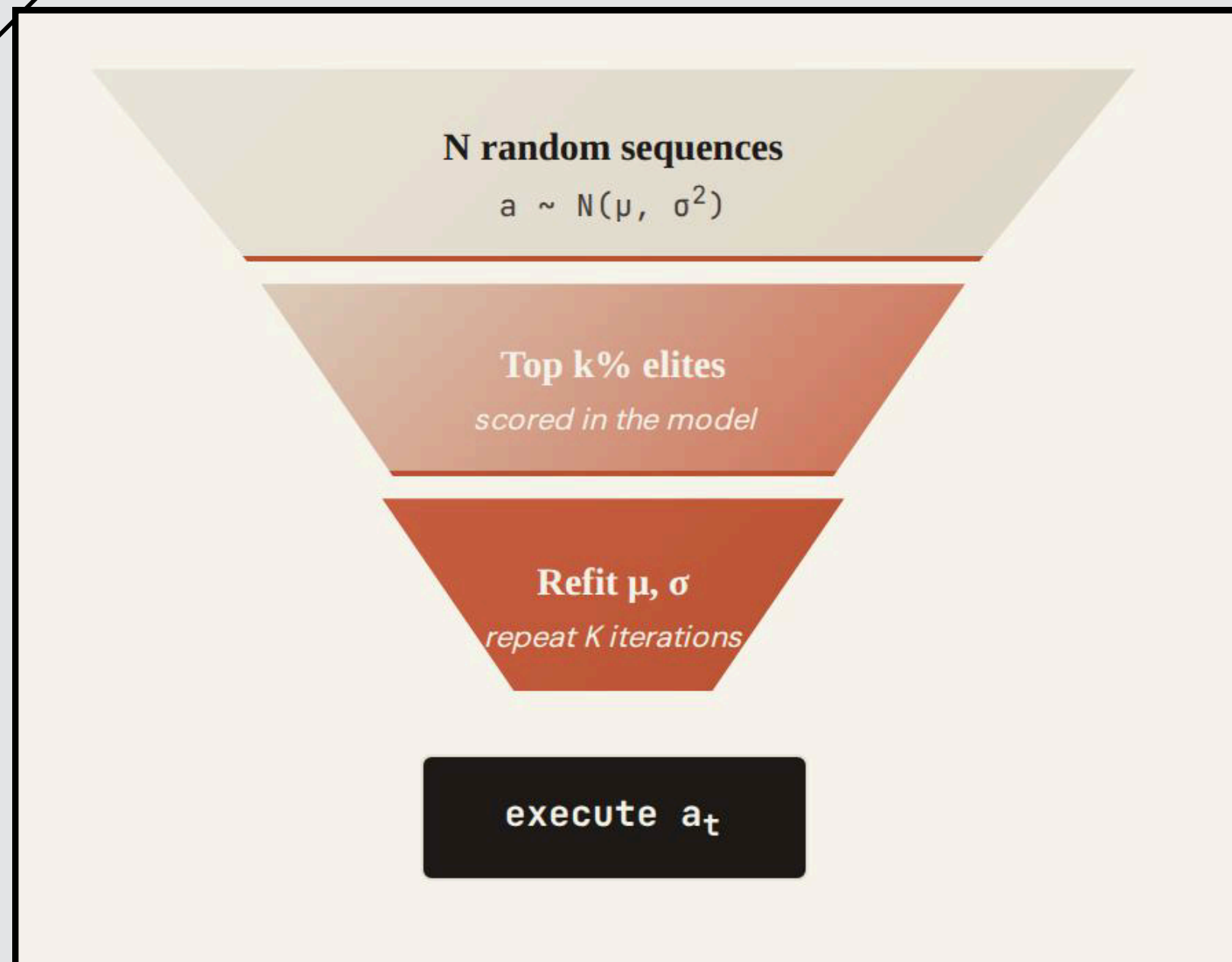
- ① Current state s_t
↓
- ② Search for the best action sequence:
 $a_t, \dots, a_{t+H}$
- ③ Execute only the first action
↓
- ④ Observe new state
↓
- ⑤ Re-plan from new state



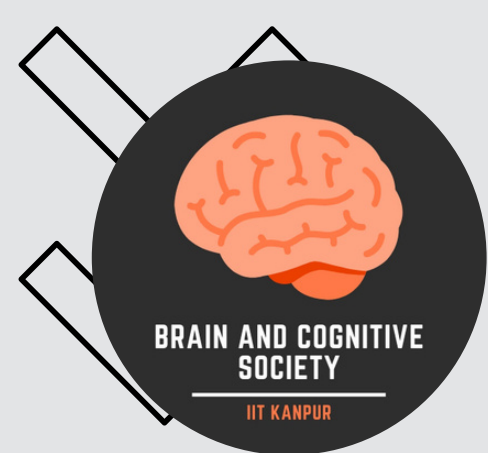


CEM: GRADIENT-FREE TRAJECTORY SEARCH

CEM is the optimizer used to solve the MPC optimization problem.



- ① Sample N candidate action sequences
(from Gaussian distribution)
- ↓
- ② Roll each sequence through the learned world model
- ↓
- ③ Compute predicted cumulative reward
- ↓
- ④ Keep top K% elite trajectories
- ↓
- ⑤ Refit Gaussian
(μ, σ)
- ↓
- Repeat K iterations
- ↓
- Execute first action



OA IMPROVEMENTS TO CEM

01

SHIFT INITIALISATION

PROBLEM

Vanilla CEM starts from scratch every timestep.



OA

Reuse the remaining H-1 actions from the previous optimal plan as the initial mean.



BENEFIT

Faster convergence and fewer optimization iterations.

02

COLORED NOISE

PROBLEM

Independent Gaussian samples produce jerky action sequences.



OA

Introduce temporal correlation between neighbouring actions.



BENEFIT

Smoother, more realistic trajectories and better exploration.

03

ELITE MEMORY

PROBLEM

Good trajectories can disappear after one iteration.



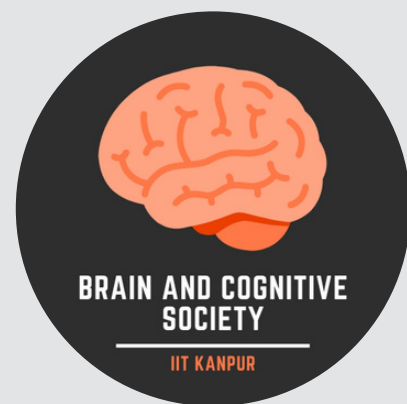
OA

Carry forward a fraction of elite candidates across iterations.



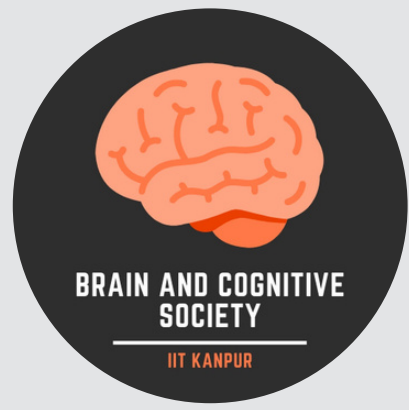
BENEFIT

Retains promising solutions and improves optimization stability.



WHAT MAKES A WORLD MODEL GOOD?

PROPERTY	WHY IT MATTERS	DEEP MLP
Accurate	Planning on a wrong model produces a wrong policy. Errors compound over horizon H .	✓
Sample-efficient	A lifelong robot cannot afford millions of training transitions per task.	✓
Fast to query	CEM calls the model $N \times H \times K$ times per real step. Latency is a hard constraint.	✓
No forgetting	Stays accurate on every task seen, while still learning the new one. This is the hard part.	✗



DEEP WORLD MODELS

Most modern model-based RL methods learn dynamics using deep neural networks.

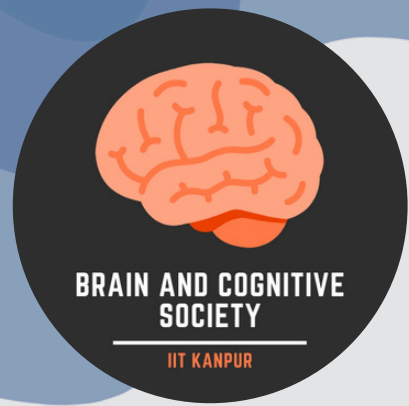
MAIN IDEA

1. **Deep neural dynamics model (MLP, ensemble or latent model):** Inputs (s, a) ; predicts next state s' or its delta.
2. **Gradient-Based Training:** The model is trained with SGD/Adam on mini-batches sampled from a replay buffer.
3. **Rich Representations:** High-capacity networks learn expressive nonlinear features capable of modeling complex dynamics.

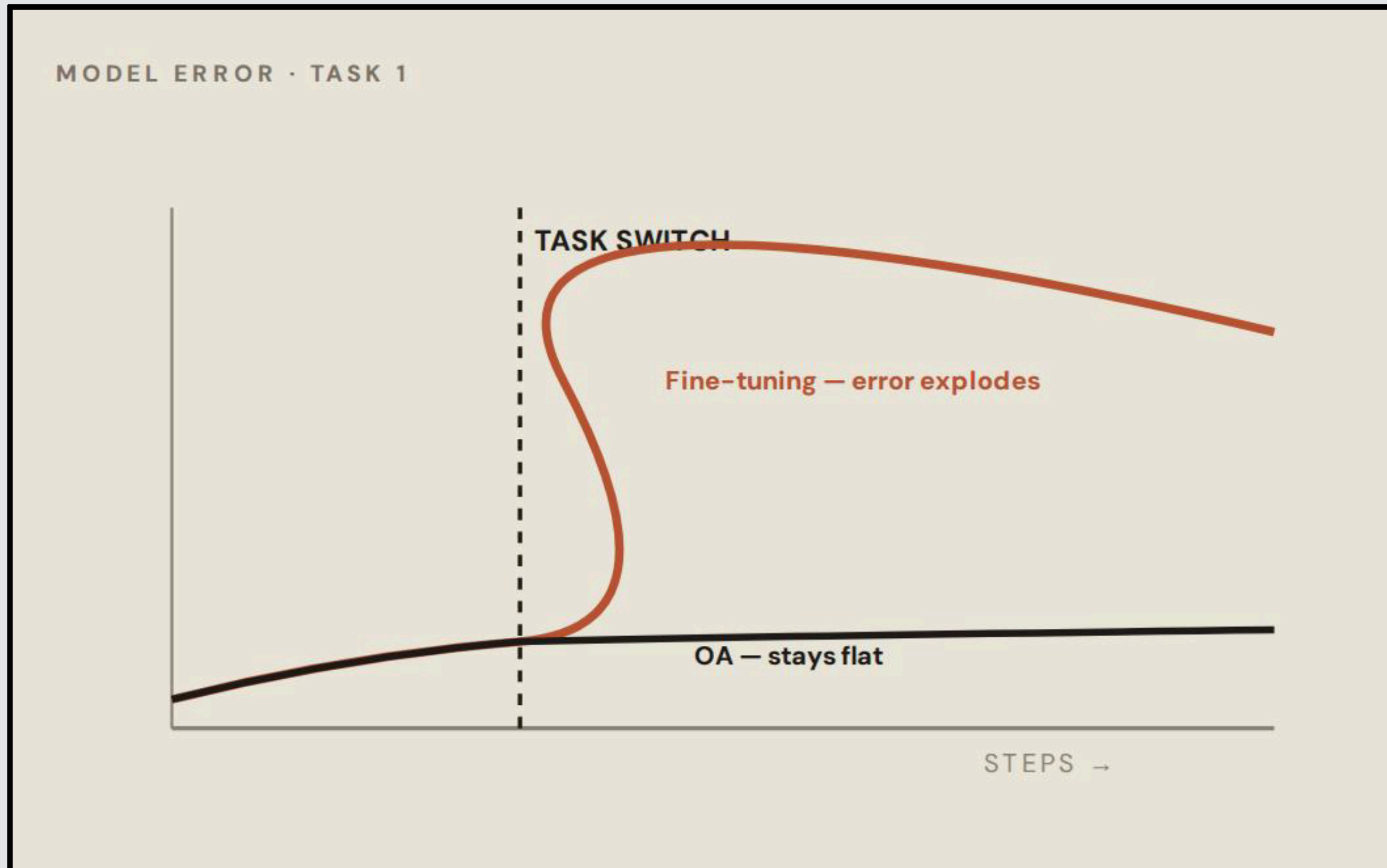
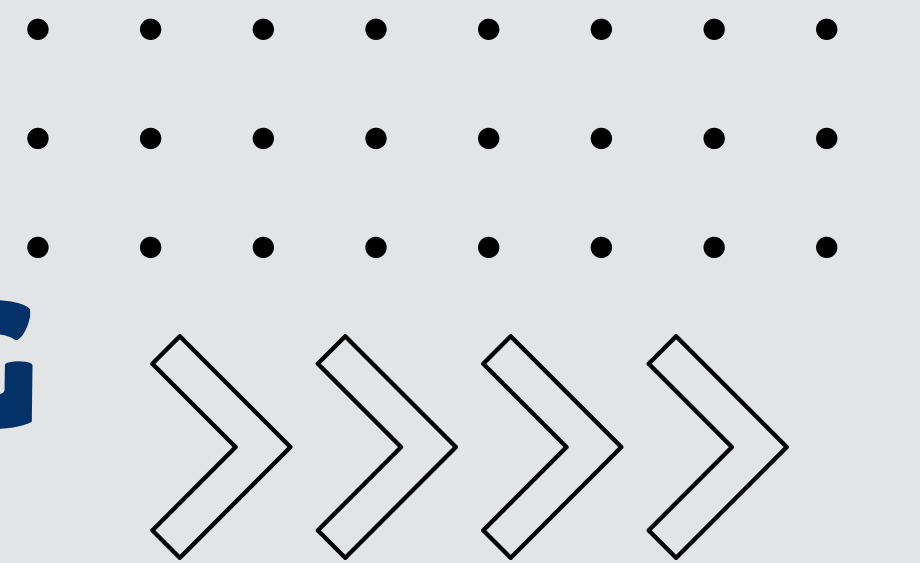
✓ *Accurate single-task performance*

✓ *Excellent single-task planning*

✗ *Continual fine-tuning causes catastrophic forgetting*

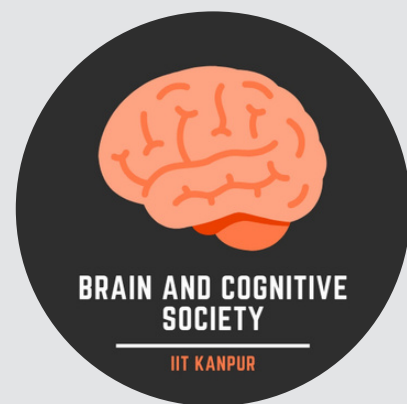


CATASTROPHIC FORGETTING



ROOT CAUSE

SGD finds the solution that fits the current data. There is nothing in the gradient that preserves old solutions. When Task 2 transitions look different from Task 1, the optimizer happily marches the weights toward Task 2, overwriting Task 1 along the way.



THREE FAMILIES: COMMON CONTINUAL-LEARNING STRATEGIES

FAMILY 1

01 Regularization

EWC · SI

MECHANISM

Penalize changes to "important" weights: importance estimated from previous tasks (e.g. Fisher Information in EWC).

FAMILY 2

02 Replay

Coreset · Perfect Memory

MECHANISM

Keep a buffer of old transitions and mix them into every gradient batch; keep showing the network the past.

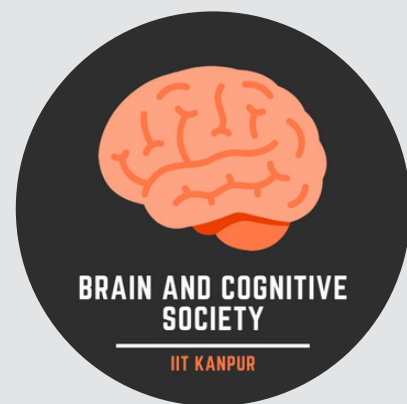
FAMILY 3

03 Architecture

PackNet · Progressive Nets

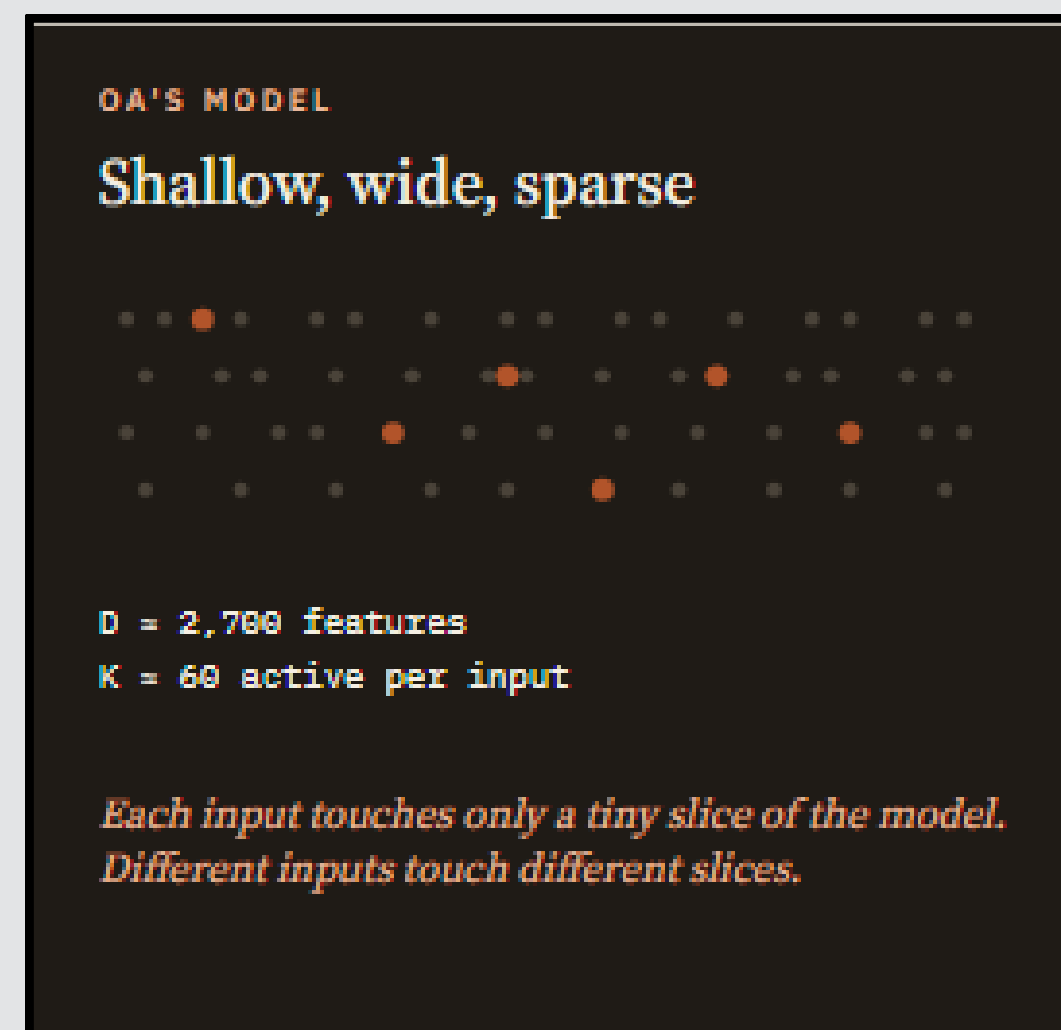
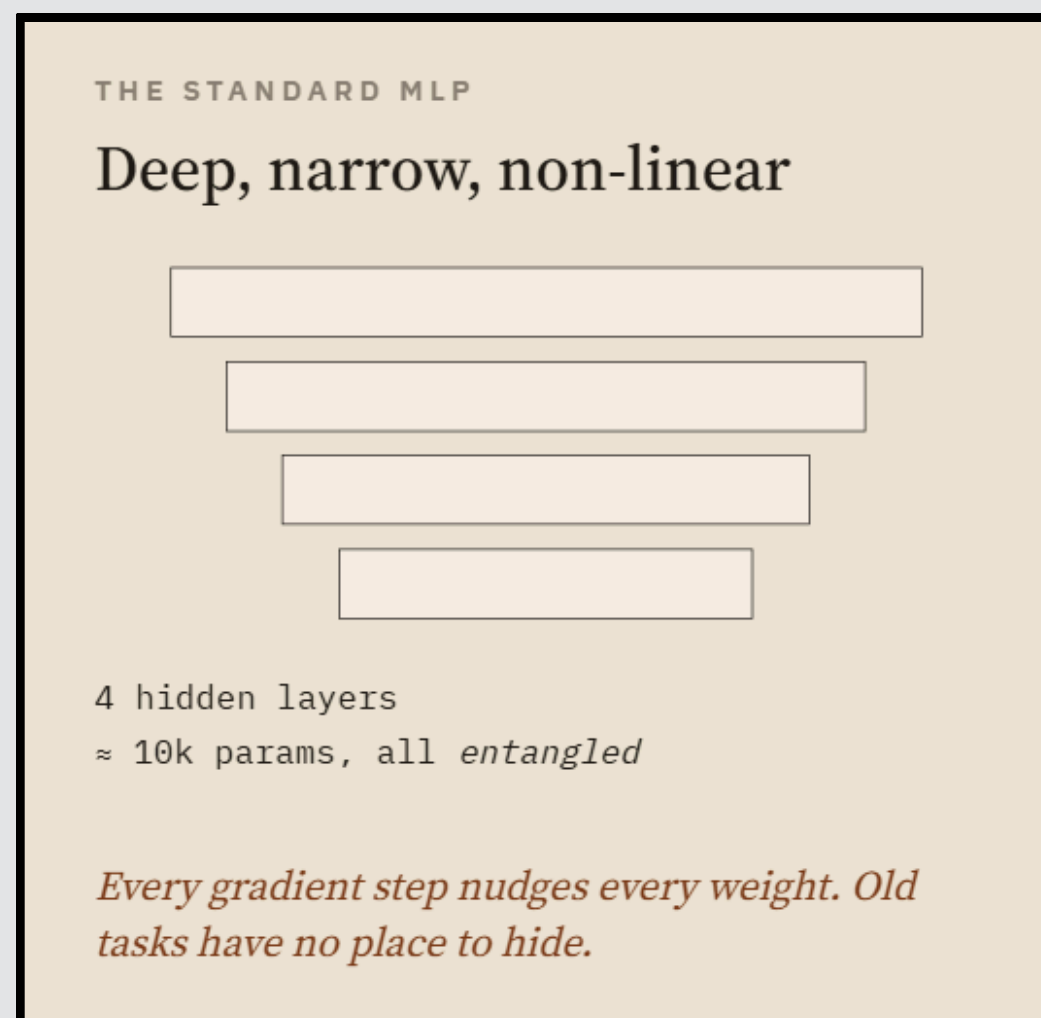
MECHANISM

Freeze a sub-network for each task; new tasks recruit fresh capacity. The old weights become read-only.



WHAT IF WE STOP USING A DEEP NETWORK?

Herein lies OA's central motivation: to replace deep nonlinear function approximation with sparse linear prediction. Indeed, our observations so far have encouraged us to consider a structural change rather than a procedural one



$$\hat{y} = W \cdot \varphi(x)$$

$x = (s, a)$ state-action pair
 $\varphi(x) \in \mathbb{R}^D$ sparse random features
 $W \in \mathbb{R}^{D \times S}$ a single linear layer
 $\hat{y} = s_{t+1} - s_t$ state delta

*One learned linear layer.
The non-linearity is all in φ .*



THE LOSSE ENCODER: A DIFFERENTIABLE RANDOM HASH

How OA builds sparse, high-dimensional features without learning any of them

PIPELINE

1. Random project the input into d dimensions:

$$\sigma(x) = \text{sigmoid}(Px), P \text{ sampled from } N(0, 1/(S+A) \cdot I)$$

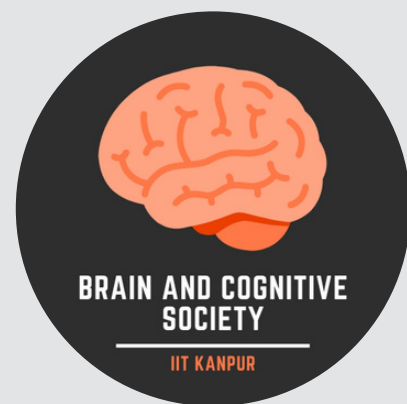
2. Softly bin each dimension into Λ bins. For each projected value, only the two neighbouring bin-edges receive non-zero weight (interpolation).

3. Concatenate: you get a sparse vector $\varphi(x) \in \mathbb{R}^D$ with $D = d \cdot \Lambda$. Exactly $K = 2d$ non-zeros, fixed, regardless of D .

4. Linear head: $\hat{y} = W \cdot \varphi(x)$; W is the only thing that's actually learned.

INTUITION

- A differentiable random hash: Similar (s, a) inputs land in nearby bins $\rightarrow$ smooth generalization. The randomness preserves distances (Johnson-Lindenstrauss).
- Increase $\Lambda \rightarrow$ wider, sparser model $\rightarrow$ more capacity.
- But $K = 2d$ stays fixed, so update cost doesn't change. Capacity grows for free



FOLLOW-THE-LEADER

No SGD. At each step, we will solve the full regularised least-squares problem on all data so far.

FTL UPDATE RULE

$$W^{(t)} = \arg \min_W \quad \|\Phi_{t-1} W - Y_{t-1}\|_F^2 + (1/\lambda) \|W\|_F^2$$

The global optimum on every observation seen since $t = 1$ — recomputed each step.

SGD Says

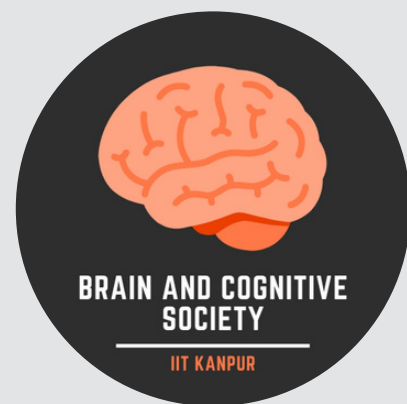
"Take a small step in the direction of the current data."

Old data is gone unless you replay it. There is a clear notion of "before" and "after"; and the after wins.

FTL Says

"Solve the whole problem, including everything I've ever seen."

No "old data" to forget, for every past observation is still in the objective. This largely eliminates forgetting.



HOWEVER..

There's a catch: solving this naively costs $O(D^2)$ per step.
Let us devise a way to solve this in $O(K^2)$ per step, forever!

THE TRICK

Never store data. Store two matrices.

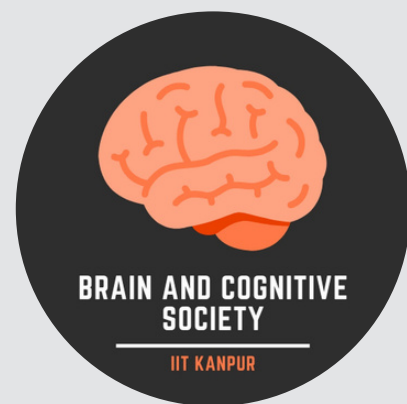
$$\begin{aligned} A^{(t)} &= \Phi^T \Phi \leftarrow \text{rank-1 add: } \phi\phi^T \\ B^{(t)} &= \Phi^T Y \leftarrow \text{rank-1 add: } \phi y^T \end{aligned}$$

A, B are fixed-size summaries of all past data. The full dataset never has to be touched again.

SPARSE UPDATE — ONLY THE K ACTIVE INDICES

$$W_s^{(t)} = (A_{ss} + I/\lambda)^{-1} \cdot (B_s - A_{ss} W_s^{(t-1)})$$

s contains all the indices on which the latest input feature $\phi(x_{t-1})$ has non-zero values, i.e., the newly coming data point only activates $K = |s| \ll D$ nodes, and $\bar{s}$ is its complement



PROVABLY NO REGRET

Empirically clean is one thing. OA is also theoretically clean; deep models have no such guarantee.

THEOREM 1 (SIMPLIFIED)

$$\text{Regret}(T) \leq O \left(\sqrt{K^2 \cdot D \cdot \log T} \right)$$

Sublinear in T — the average per-step loss vs. the best fixed model goes to 0.

What "regret" means here:

The total excess loss of the online learner vs. the best fixed linear model with hindsight.

Per step $\rightarrow 0 \Rightarrow$ OA converges to the model you'd have fit if you had all the data from the start.

Reading the bound:

$\sqrt{D}$ model complexity
 K^2 sparsity-approximation cost
 $\log T$ the gentlest growth in T

*No dependence on the number of tasks.
Switching tasks doesn't penalize the bound.*

Three mild assumptions

- A1. Feature covariance stabilizes
- A2. Inputs and outputs are bounded
- A3. K is large enough relative to feature statistics

None of these are exotic: they hold for bounded robotic state spaces and a wide enough Λ .



THE COMPLETE OA LOOP

One model, one planner, one rule. No replay, no task IDs, no growing memory.

THE ALGORITHM

loop forever:

① update $W^{(t)}$ via sparse FTL $[O(K^2)]$

② plan with CEM using $W^{(t)} \rightarrow a_t$

③ execute $a_t \rightarrow$ observe s_{t+1}

④ $x_t = (s_t, a_t), y_t = s_{t+1} - s_t$

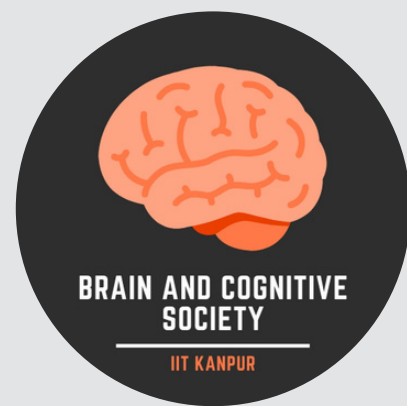
⑤ $A \ += \ \phi(x_t) \phi(x_t)^T \quad [O(K^2)]$

⑥ $B \ += \ \phi(x_t) y_t^T \quad [O(K)]$

⑦ if task changes: reset CEM μ

What this loop does NOT need?

- **No replay buffer:** Past data lives compressed inside A and B forever.
- **No task boundaries:** The world model's update doesn't care when the task changes.
- **No growing cost:** Per-step compute is constant — the same on day 1 and day 1000.
- **Reward is plug-and-play:** New task = new reward handed to CEM. World model untouched.



OA MATCHES PERFECT MEMORY – AT A FRACTION OF THE COST

Same final performance, lower regret, constant per-step compute.

METHOD	AP ↑	REG ↓
Fine-tuning	24.86%	37.74%
SI	39.96%	33.57%
Coreset	61.83%	30.83%
Perfect Memory <i>retrains on all past data, every step</i>	73.09%	30.95%
OA (<i>ours</i>) <i>constant $O(K^2)$ per step</i>	72.93%	27.62%

THREE FINDINGS

① Ties Perfect Memory on AP.

A single linear layer matches the gold-standard replay method.

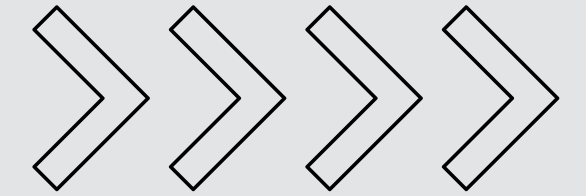
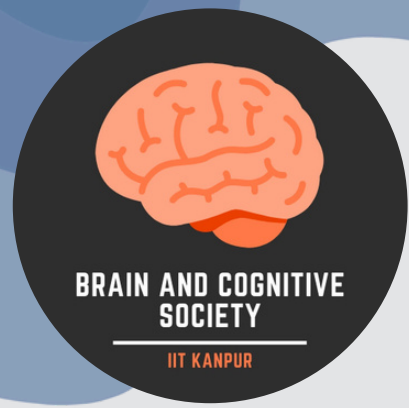
② Beats it on regret.

An exact LS solution > an SGD approximation, even with full data.

③ At constant cost.

Perfect Memory's compute scales with task count. OA's doesn't move.

AP = average final performance across all tasks. Reg = cumulative regret across the lifetime.



THANK YOU!

